Day 6 — Multi-Agent Collaboration

> 🎯 **Goal**: Build a system where multiple Agents work together intelligently.

> ⏱️ **Time**: 2h theory + 3h practice

> 📦 **Deliverable**: A multi-agent system solving engineering tasks end-to-end

---

Table of Contents

1. Why Multi-Agent?
2. Orchestration Patterns
3. Agent Roles
4. Communication & Turn-Taking
5. Preventing Loops & Deadlocks
6. Practice: Build a 3-Agent System

Why Multi-Agent?

A single agent has limits. It must be good at planning AND executing AND validating AND researching. Multi-agent systems let you **specialize** — each agent is an expert at one thing.



```
   | Expert at|  | Expert at|  | Expert at|
   | breaking |  | running  |  | finding  |
   | down     |  | tools &  |  | bugs &   |
   | tasks    |  | writing  |  | quality  |
   |          |  | code     |  | issues   |
   └──────────┘  └──────────┘  └──────────┘
        |             |             |
        └─────────────┼─────────────┘
                      |
              ORCHESTRATOR
              (coordinates)

Benefits:
  ✓ Focused prompts → better results per role
  ✓ Different models per role (GPT-4 for planning, GPT-3.5 for execution)
  ✓ Parallel execution where possible
  ✓ Clear separation of concerns
  ✓ Easier to debug (which agent went wrong?)
```

Orchestration Patterns

Pattern 1: Sequential Pipeline

```
Agents process in a fixed order, like an assembly line.

  Task → Planner → Executor → Critic → Output
                                 |
                          (retry if failed)

Best for: Well-defined workflows with clear stages
Example: Code generation (plan → code → review)

Pros: Simple, predictable, easy to debug
Cons: Rigid, can't handle dynamic task shapes
```

Pattern 2: Coordinator (Hub-and-Spoke)

```
A central Coordinator agent decides which specialist to call next.

                    ┌──────────────┐
                    │ COORDINATOR  │
                    │              │
                    │ "Who should  │
                    │  handle this │
                    │  next?"      │
                    └──────┬───────┘
                           |
                ┌──────────┼──────────┐
```

```
            ▼           ▼           ▼
       ┌──────────┐ ┌──────────┐ ┌──────────┐
       │RESEARCHER│ │  CODER   │ │  CRITIC  │
       └──────────┘ └──────────┘ └──────────┘

Flow:
  1. Coordinator receives task
  2. Coordinator decides: "First, we need research"
  3. Researcher does research, returns results
  4. Coordinator decides: "Now we need code"
  5. Coder writes code using research
  6. Coordinator decides: "Now we need review"
  7. Critic reviews code
  8. Coordinator: "Critic found issues → back to Coder"
  9. ...until Coordinator is satisfied

Best for: Complex, dynamic tasks
Example: "Given a Jira ticket, solve it end-to-end"

Pros: Flexible, adaptive routing
Cons: Coordinator becomes bottleneck, needs good judgment
```

Pattern 3: Debate / Consensus

```
Multiple agents tackle the same problem independently,
then debate/vote on the best solution.

  Task ──┬── Agent A ──┐
         ├── Agent B ──┼── Debate → Consensus → Output
         └── Agent C ──┘

Best for: High-stakes decisions, diverse perspectives
Example: Security analysis, architectural decisions

Pros: Reduces individual agent bias, higher quality
Cons: Expensive (N× the LLM calls), slower
```

Pattern 4: Hierarchical Teams

```
Agents organized in teams with team leads.

  ┌──────────────────────────────────────────────┐
  │              PROJECT MANAGER                 │
  │         (top-level coordinator)              │
  └──────────┬──────────────────┬────────────────┘
             ▼                  ▼
  ┌──────────────────┐  ┌──────────────────┐
  │ BACKEND TEAM     │  │ FRONTEND TEAM    │
  │ Lead             │  │ Lead             │
  │  ├── Coder       │  │  ├── Designer    │
  │  ├── Tester      │  │  ├── Coder       │
```

```
   │  └── Reviewer    │  │  └── Tester      │
   └──────────────────┘  └──────────────────┘

Best for: Large, complex projects
Example: Building a full application

Pros: Scales to many agents, clear accountability
Cons: Complex, high overhead, hard to debug
```

Choosing a Pattern

```
┌──────────────────────┬──────────────────────┬──────────────┐
│ Pattern              │ Best For             │ Complexity   │
├──────────────────────┼──────────────────────┼──────────────┤
│ Sequential Pipeline  │ Predictable workflows  │ Low        │
│ Coordinator          │ Dynamic, complex tasks │ Medium     │
│ Debate/Consensus     │ High-stakes decisions  │ Medium     │
│ Hierarchical Teams   │ Large projects         │ High       │
└──────────────────────┴──────────────────────┴──────────────┘

Start with Sequential Pipeline. Move to Coordinator when needed.
```

Agent Roles

Common Agent Archetypes

```
┌──────────────────────────────────────────────────────────┐
│                    AGENT ROLES                           │
│                                                          │
│  PLANNER                                                 │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Responsibility: Break tasks into steps, create plans  │
│  │ Skills: Task decomposition, dependency analysis  │    │
│  │ Model: Needs strong reasoning (GPT-4, Claude 3)  │    │
│  │ Temperature: Low (0.0-0.2)                       │    │
│  │ Output: Structured plan (JSON)                   │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  RESEARCHER                                              │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Responsibility: Gather information, search knowledge   │
│  │ Skills: Query formulation, source evaluation     │    │
│  │ Tools: Search, RAG, web browsing                 │    │
│  │ Model: Medium (GPT-3.5 is fine)                  │    │
│  │ Output: Summarized findings with sources         │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  CODER                                                   │
```

```
│  ┌──────────────────────────────────────────────────────┐  │
│  │ Responsibility: Write, modify, and fix code          │  │
│  │ Skills: Programming, debugging, refactoring          │  │
│  │ Tools: Code execution, file I/O, package managers    │  │
│  │ Model: Coding-optimized (GPT-4, Claude, Codex)       │  │
│  │ Output: Working code with comments                   │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  CRITIC                                                    │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ Responsibility: Review, validate, find issues        │  │
│  │ Skills: Bug detection, quality assessment            │  │
│  │ Tools: Code analysis, test runners                   │  │
│  │ Model: Needs judgment (GPT-4)                        │  │
│  │ Output: Issues list with verdicts (PASS/RETRY/REPLAN)│  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
│  TOOL EXECUTOR                                             │
│  ┌──────────────────────────────────────────────────────┐  │
│  │ Responsibility: Run tools safely, return results     │  │
│  │ Skills: Tool orchestration, error handling           │  │
│  │ Tools: All external tools (APIs, databases, etc.)    │  │
│  │ Model: Light (GPT-3.5) or deterministic (no LLM)     │  │
│  │ Output: Tool execution results                       │  │
│  └──────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────┘
```

System Prompt Design for Each Role

```
AGENT_PROMPTS = {
    "planner": """You are an expert task planner. Your ONLY job is to break
down complex tasks into clear, ordered steps.

Rules:
- Each step must be concrete and actionable
- Include dependencies between steps
- Include validation criteria for each step
- Do NOT execute steps — only plan them
- Output a JSON plan

You have these agents available to assign steps to:
- researcher: gathers information
- coder: writes and modifies code
- critic: reviews and validates work
""",

    "researcher": """You are a research specialist. Your ONLY job is to
gather relevant information for a given topic.

Rules:
- Search thoroughly before answering
- Always cite your sources
```

```
- Distinguish facts from opinions
- If information is insufficient, say so clearly
- Output structured findings as JSON
""",

    "coder": """You are a senior software engineer. Your ONLY job is to
write clean, working code.

Rules:
- Follow best practices for the language
- Include error handling
- Add comments for complex logic only
- Write testable code
- If requirements are unclear, list your assumptions
""",

    "critic": """You are a code review and quality assurance expert.
Your ONLY job is to evaluate work quality.

Rules:
- Check for bugs, security issues, and logic errors
- Verify requirements are met
- Be specific about what's wrong and how to fix it
- Rate issues as CRITICAL, HIGH, MEDIUM, LOW
- Respond with PASS, RETRY, or REPLAN verdict
- Do NOT fix issues yourself — only identify them
""",
}
```

Communication & Turn-Taking

Message Protocol

Agents need a standard way to communicate:

```python
from dataclasses import dataclass, field
from typing import Optional
from enum import Enum
import time

class MessageType(Enum):
    TASK = "task"           # Assign a task
    RESULT = "result"       # Return results
    FEEDBACK = "feedback"   # Provide feedback
    QUESTION = "question"   # Ask for clarification
    STATUS = "status"       # Status update

@dataclass
class AgentMessage:
```

```
        """Standard message format between agents."""
        from_agent: str
        to_agent: str
        type: MessageType
        content: str
        metadata: dict = field(default_factory=dict)
        timestamp: float = field(default_factory=time.time)
        reply_to: Optional[str] = None  # ID of message this replies to
        id: str = field(default_factory=lambda: f"msg_{int(time.time()*1000)}")

        def to_dict(self) -> dict:
            return {
                "id": self.id,
                "from": self.from_agent,
                "to": self.to_agent,
                "type": self.type.value,
                "content": self.content,
                "metadata": self.metadata,
                "timestamp": self.timestamp,
                "reply_to": self.reply_to
            }
```

Turn-Taking Strategies

```
Strategy 1: ROUND-ROBIN
  Each agent takes a turn in fixed order.
  Planner → Executor → Critic → Planner → ...

  Simple but inflexible.

Strategy 2: COORDINATOR-DRIVEN
  Coordinator decides who speaks next based on current state.

  "Plan exists?" → No → Planner
  "Step to execute?" → Yes → Executor
  "Result to validate?" → Yes → Critic

  Flexible and adaptive.

Strategy 3: EVENT-DRIVEN
  Agents react to events (messages) asynchronously.

  Planner publishes "plan_ready" → Executor picks it up
  Executor publishes "step_done" → Critic picks it up
  Critic publishes "issues_found" → Coder picks it up

  Most scalable but hardest to debug.
```

Preventing Loops & Deadlocks

Common Failure Modes

```
FAILURE 1: Agent Ping-Pong
  Coder: "Here's the code"
  Critic: "This has issues, please fix"
  Coder: "Here's the fixed code" (same code!)
  Critic: "This still has issues"
  Coder: "Here's the fixed code" (same code!)
  ... infinite ...

  Prevention:
  - Track what feedback was given
  - After N retries, escalate or change approach
  - Include previous attempt in context

FAILURE 2: Deadlock
  Agent A: "Waiting for Agent B's output"
  Agent B: "Waiting for Agent A's output"
  ... both stuck forever ...

  Prevention:
  - No circular dependencies in the plan
  - Timeout per agent (e.g., 60 seconds)
  - Deadlock detection: if no progress for N seconds, break

FAILURE 3: Cascading Failures
  Planner creates bad plan → Executor fails on step 1
  → Critic says REPLAN → Planner creates same bad plan
  → Executor fails again → ...

  Prevention:
  - Include failure history in replan context
  - Max replan limit (e.g., 3)
  - Different approach after each failure

FAILURE 4: Token Budget Exhaustion
  Each agent call uses tokens. Multi-agent uses N× more.
  Long conversations can exceed context windows.

  Prevention:
  - Summarize inter-agent messages
  - Set per-agent token budgets
  - Use lighter models for simple agents
```

Guardrail Implementation

```python
  @dataclass
  class MultiAgentGuardrails:
      """Guardrails for multi-agent systems."""
      max_total_turns: int = 30
      max_turns_per_agent: int = 10
      max_retries_per_step: int = 3
```

```
    max_replans: int = 2
    timeout_per_turn_seconds: float = 60.0

    # Tracking
    turn_count: int = 0
    agent_turn_counts: dict = field(default_factory=dict)
    retry_counts: dict = field(default_factory=dict)
    replan_count: int = 0

    def check_turn(self, agent_name: str) -> tuple:
        """Check if this agent can take another turn."""
        self.turn_count += 1
        self.agent_turn_counts[agent_name] = \
            self.agent_turn_counts.get(agent_name, 0) + 1

        if self.turn_count > self.max_total_turns:
            return False, f"Total turn limit ({self.max_total_turns}) reached"

        if self.agent_turn_counts[agent_name] > self.max_turns_per_agent:
            return False, f"{agent_name} turn limit ({self.max_turns_per_agent}) rea

        return True, ""

    def check_retry(self, step_id: str) -> tuple:
        """Check if a step can be retried."""
        self.retry_counts[step_id] = self.retry_counts.get(step_id, 0) + 1

        if self.retry_counts[step_id] > self.max_retries_per_step:
            return False, f"Retry limit for step '{step_id}' reached"

        return True, ""

    def check_replan(self) -> tuple:
        """Check if replanning is allowed."""
        self.replan_count += 1
        if self.replan_count > self.max_replans:
            return False, f"Replan limit ({self.max_replans}) reached"
        return True, ""
```

Practice: Build a 3-Agent System

Complete Multi-Agent Implementation

```
"""
Multi-Agent System: Planner + Executor + Critic

Solves engineering tasks by:
1. Planner breaks down the task
2. Executor performs each step
```

```
    3. Critic validates each result

    Real-world task: "Given a Jira ticket, propose a solution,
    generate the code, test it, and provide a PR summary."

    No API key needed — fully functional with simulated LLM responses.
    """
    import json
    import time
    from dataclasses import dataclass, field
    from typing import Optional
    from enum import Enum


    # ═══════════════════════════════════════════════════════════
    # MESSAGE SYSTEM
    # ═══════════════════════════════════════════════════════════

    class Role(Enum):
        ORCHESTRATOR = "orchestrator"
        PLANNER = "planner"
        EXECUTOR = "executor"
        CRITIC = "critic"

    @dataclass
    class Message:
        """Inter-agent message."""
        sender: Role
        receiver: Role
        content: str
        msg_type: str  # "task", "result", "feedback", "verdict"
        data: dict = field(default_factory=dict)
        timestamp: float = field(default_factory=time.time)

    class MessageBus:
        """Central message bus for agent communication."""

        def __init__(self):
            self.messages: list[Message] = []
            self.subscribers: dict = {}

        def send(self, message: Message):
            """Send a message."""
            self.messages.append(message)

        def get_messages_for(self, role: Role,
                             msg_type: Optional[str] = None) -> list:
            """Get all messages for a specific agent."""
            msgs = [m for m in self.messages if m.receiver == role]
            if msg_type:
                msgs = [m for m in msgs if m.msg_type == msg_type]
            return msgs

        def get_conversation_log(self) -> str:
            """Get a formatted log of all messages."""
            log = []
```

```
        for msg in self.messages:
            log.append(
                f"[{msg.sender.value} → {msg.receiver.value}] "
                f"({msg.msg_type}) {msg.content[:100]}"
            )
        return "\n".join(log)


# ═══════════════════════════════════════════════════════════════
# AGENT BASE CLASS
# ═══════════════════════════════════════════════════════════════

class BaseAgent:
    """Base class for all agents."""

    def __init__(self, role: Role, bus: MessageBus):
        self.role = role
        self.bus = bus
        self.memory = []  # Agent's internal memory

    def send(self, to: Role, content: str, msg_type: str,
             data: dict = None):
        """Send a message to another agent."""
        self.bus.send(Message(
            sender=self.role,
            receiver=to,
            content=content,
            msg_type=msg_type,
            data=data or {}
        ))

    def log(self, message: str):
        """Log agent activity."""
        prefix = {
            Role.ORCHESTRATOR: "🎯",
            Role.PLANNER: "📋",
            Role.EXECUTOR: "⚡",
            Role.CRITIC: "🔍"
        }.get(self.role, "•")
        print(f"  {prefix} [{self.role.value}] {message}")


# ═══════════════════════════════════════════════════════════════
# PLANNER AGENT
# ═══════════════════════════════════════════════════════════════

class PlannerAgent(BaseAgent):
    """
    Creates structured plans from task descriptions.
    Specializes in task decomposition and dependency analysis.
    """

    def __init__(self, bus: MessageBus):
        super().__init__(Role.PLANNER, bus)

    def create_plan(self, task: str, context: str = "") -> dict:
```

```
        """Create a plan for the given task."""
        self.log(f"Creating plan for: {task[:80]}...")

        plan = self._decompose_task(task, context)

        self.log(f"Plan created: {len(plan['steps'])} steps")
        for step in plan["steps"]:
            self.log(f"  Step {step['id']}: {step['description']}")

        self.send(
            Role.ORCHESTRATOR,
            f"Plan created with {len(plan['steps'])} steps",
            "result",
            {"plan": plan}
        )

        return plan

    def revise_plan(self, original_plan: dict, feedback: str,
                    completed_steps: list) -> dict:
        """Revise a plan based on critic feedback."""
        self.log(f"Revising plan based on feedback: {feedback[:80]}...")

        # Keep completed steps, add recovery + remaining
        new_steps = []
        next_id = 1

        for step in original_plan["steps"]:
            if step["id"] in completed_steps:
                step["status"] = "done"
                step["id"] = next_id
                new_steps.append(step)
                next_id += 1

        # Add recovery step
        new_steps.append({
            "id": next_id,
            "description": f"Address feedback: {feedback}",
            "agent": "executor",
            "action": "fix",
            "inputs": {"feedback": feedback},
            "status": "pending"
        })
        next_id += 1

        # Add validation step
        new_steps.append({
            "id": next_id,
            "description": "Validate the fix addresses the feedback",
            "agent": "critic",
            "action": "validate",
            "inputs": {},
            "status": "pending"
        })

        revised = {
```

```
            "task": original_plan["task"],
            "approach": f"Revised: {feedback}",
            "steps": new_steps,
            "version": original_plan.get("version", 1) + 1
        }

        self.log(f"Revised plan: {len(new_steps)} steps (v{revised['version']})")
        return revised

    def _decompose_task(self, task: str, context: str) -> dict:
        """Break a task into steps (rule-based for demo)."""
        task_lower = task.lower()

        # Detect task type and create appropriate plan
        if "jira" in task_lower or "ticket" in task_lower or "implement" in task_low
            return self._plan_engineering_task(task)
        elif "analyze" in task_lower or "review" in task_lower:
            return self._plan_review_task(task)
        else:
            return self._plan_generic_task(task)

    def _plan_engineering_task(self, task: str) -> dict:
        return {
            "task": task,
            "approach": "Analyze → Design → Implement → Test → Review → PR Summary",
            "steps": [
                {
                    "id": 1,
                    "description": "Analyze the ticket requirements and identify sco
                    "agent": "executor",
                    "action": "analyze",
                    "inputs": {"task": task},
                    "status": "pending"
                },
                {
                    "id": 2,
                    "description": "Research best practices and similar implementati
                    "agent": "executor",
                    "action": "research",
                    "inputs": {"topic": task},
                    "status": "pending"
                },
                {
                    "id": 3,
                    "description": "Generate the implementation code",
                    "agent": "executor",
                    "action": "code",
                    "inputs": {},
                    "status": "pending"
                },
                {
                    "id": 4,
                    "description": "Write unit tests for the implementation",
                    "agent": "executor",
                    "action": "test",
                    "inputs": {},
```

```
                    "status": "pending"
                },
                {
                    "id": 5,
                    "description": "Review code quality and correctness",
                    "agent": "critic",
                    "action": "review",
                    "inputs": {},
                    "status": "pending"
                },
                {
                    "id": 6,
                    "description": "Generate PR summary with changes and test result
                    "agent": "executor",
                    "action": "summarize",
                    "inputs": {},
                    "status": "pending"
                }
            ],
            "version": 1
        }

    def _plan_review_task(self, task: str) -> dict:
        return {
            "task": task,
            "approach": "Gather → Analyze → Report",
            "steps": [
                {"id": 1, "description": "Gather materials to review",
                 "agent": "executor", "action": "research",
                 "inputs": {"topic": task}, "status": "pending"},
                {"id": 2, "description": "Perform detailed analysis",
                 "agent": "executor", "action": "analyze",
                 "inputs": {}, "status": "pending"},
                {"id": 3, "description": "Validate analysis quality",
                 "agent": "critic", "action": "review",
                 "inputs": {}, "status": "pending"},
                {"id": 4, "description": "Compile final report",
                 "agent": "executor", "action": "summarize",
                 "inputs": {}, "status": "pending"},
            ],
            "version": 1
        }

    def _plan_generic_task(self, task: str) -> dict:
        return {
            "task": task,
            "approach": "Research → Execute → Validate",
            "steps": [
                {"id": 1, "description": f"Research: {task}",
                 "agent": "executor", "action": "research",
                 "inputs": {"topic": task}, "status": "pending"},
                {"id": 2, "description": "Execute the main task",
                 "agent": "executor", "action": "execute",
                 "inputs": {}, "status": "pending"},
                {"id": 3, "description": "Validate the result",
                 "agent": "critic", "action": "review",
```

```
                "inputs": {}, "status": "pending"},
            ],
            "version": 1
        }


# ============================================================
# EXECUTOR AGENT
# ============================================================

class ExecutorAgent(BaseAgent):
    """
    Executes plan steps using available tools.
    Specializes in research, coding, testing, and summarization.
    """

    def __init__(self, bus: MessageBus):
        super().__init__(Role.EXECUTOR, bus)
        self.context = {}  # Accumulated context from previous steps

    def execute_step(self, step: dict) -> dict:
        """Execute a single step from the plan."""
        action = step.get("action", "execute")
        self.log(f"Executing: {step['description']}")

        # Route to appropriate handler
        handlers = {
            "analyze": self._analyze,
            "research": self._research,
            "code": self._write_code,
            "test": self._write_tests,
            "summarize": self._summarize,
            "fix": self._fix,
            "execute": self._generic_execute,
        }

        handler = handlers.get(action, self._generic_execute)
        result = handler(step)

        # Store in context for later steps
        self.context[step["id"]] = result

        self.log(f"Completed: {result['status']}")

        self.send(
            Role.ORCHESTRATOR,
            f"Step {step['id']} completed: {result['status']}",
            "result",
            {"step_id": step["id"], "result": result}
        )

        return result

    def _analyze(self, step: dict) -> dict:
        task = step.get("inputs", {}).get("task", "Unknown task")
        return {
```

```
            "status": "success",
            "output": (
                f"Requirements Analysis:\n"
                f"- Task: {task}\n"
                f"- Scope: Implement the requested feature\n"
                f"- Dependencies: Standard library, FastAPI\n"
                f"- Estimated complexity: Medium\n"
                f"- Key considerations: Error handling, input validation, tests"
            ),
            "artifacts": {"requirements": task}
        }

    def _research(self, step: dict) -> dict:
        topic = step.get("inputs", {}).get("topic", "")
        return {
            "status": "success",
            "output": (
                f"Research Findings:\n"
                f"- Best practice: Use FastAPI with Pydantic models for validation\n
                f"- Pattern: Repository pattern for data access layer\n"
                f"- Error handling: Use HTTPException with proper status codes\n"
                f"- Testing: Use pytest with TestClient for API testing\n"
                f"- Security: Input validation, parameterized queries, auth middlewa
            ),
            "artifacts": {"research_notes": f"Research on: {topic}"}
        }

    def _write_code(self, step: dict) -> dict:
        # Use context from previous steps
        requirements = ""
        for ctx in self.context.values():
            if isinstance(ctx, dict) and "output" in ctx:
                requirements += ctx["output"] + "\n"

        code = '''from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import Optional
import uuid

app = FastAPI(title="Task API")

class Task(BaseModel):
    title: str = Field(..., min_length=1, max_length=200)
    description: Optional[str] = None
    status: str = Field(default="todo", pattern="^(todo|in_progress|done)$")

class TaskResponse(Task):
    id: str

# In-memory store (use database in production)
tasks_db: dict[str, TaskResponse] = {}

@app.post("/tasks", response_model=TaskResponse, status_code=201)
async def create_task(task: Task):
    task_id = str(uuid.uuid4())
    task_response = TaskResponse(id=task_id, **task.model_dump())
```

```
    tasks_db[task_id] = task_response
    return task_response

@app.get("/tasks/{task_id}", response_model=TaskResponse)
async def get_task(task_id: str):
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="Task not found")
    return tasks_db[task_id]

@app.get("/tasks", response_model=list[TaskResponse])
async def list_tasks(status: Optional[str] = None):
    tasks = list(tasks_db.values())
    if status:
        tasks = [t for t in tasks if t.status == status]
    return tasks

@app.put("/tasks/{task_id}", response_model=TaskResponse)
async def update_task(task_id: str, task: Task):
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="Task not found")
    updated = TaskResponse(id=task_id, **task.model_dump())
    tasks_db[task_id] = updated
    return updated

@app.delete("/tasks/{task_id}", status_code=204)
async def delete_task(task_id: str):
    if task_id not in tasks_db:
        raise HTTPException(status_code=404, detail="Task not found")
    del tasks_db[task_id]
'''

        return {
            "status": "success",
            "output": f"Generated implementation code:\n```python\n{code}\n```",
            "artifacts": {"code": code, "filename": "main.py"}
        }

    def _write_tests(self, step: dict) -> dict:
        test_code = '''import pytest
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_create_task():
    response = client.post("/tasks", json={
        "title": "Test task",
        "description": "A test task"
    })
    assert response.status_code == 201
    data = response.json()
    assert data["title"] == "Test task"
    assert "id" in data

def test_get_task():
    # Create first
```

```
    create_resp = client.post("/tasks", json={"title": "Get test"})
    task_id = create_resp.json()["id"]

    # Then get
    response = client.get(f"/tasks/{task_id}")
    assert response.status_code == 200
    assert response.json()["title"] == "Get test"

def test_get_nonexistent_task():
    response = client.get("/tasks/nonexistent-id")
    assert response.status_code == 404

def test_list_tasks():
    response = client.get("/tasks")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_update_task():
    create_resp = client.post("/tasks", json={"title": "Update me"})
    task_id = create_resp.json()["id"]

    response = client.put(f"/tasks/{task_id}", json={
        "title": "Updated title",
        "status": "in_progress"
    })
    assert response.status_code == 200
    assert response.json()["title"] == "Updated title"

def test_delete_task():
    create_resp = client.post("/tasks", json={"title": "Delete me"})
    task_id = create_resp.json()["id"]

    response = client.delete(f"/tasks/{task_id}")
    assert response.status_code == 204

def test_invalid_status():
    response = client.post("/tasks", json={
        "title": "Bad status",
        "status": "invalid"
    })
    assert response.status_code == 422  # Validation error
'''

        return {
            "status": "success",
            "output": f"Generated test suite:\n```python\n{test_code}\n```\n\n7 test
            "artifacts": {"test_code": test_code, "test_count": 7, "filename": "test
        }

    def _summarize(self, step: dict) -> dict:
        # Compile all context into a PR summary
        artifacts = {}
        for ctx in self.context.values():
            if isinstance(ctx, dict) and "artifacts" in ctx:
                artifacts.update(ctx["artifacts"])
```

```
        summary = f"""## PR Summary

### Changes
- Implemented CRUD API for Task management using FastAPI
- Added Pydantic models with validation (title length, status enum)
- In-memory storage with full CRUD operations
- Proper error handling with HTTP status codes

### Files Changed
- `main.py` — API implementation with 5 endpoints
- `test_main.py` — Test suite with 7 tests

### Testing
- ✅ Create task (201)
- ✅ Get task (200)
- ✅ Get nonexistent (404)
- ✅ List tasks (200)
- ✅ Update task (200)
- ✅ Delete task (204)
- ✅ Invalid status validation (422)

### Considerations
- Using in-memory store; should migrate to database for production
- No authentication implemented yet
- No pagination on list endpoint
"""

        return {
            "status": "success",
            "output": summary,
            "artifacts": {"pr_summary": summary}
        }

    def _fix(self, step: dict) -> dict:
        feedback = step.get("inputs", {}).get("feedback", "")
        return {
            "status": "success",
            "output": f"Applied fix based on feedback: {feedback}\n"
                      f"Changes made: Added input validation, improved error messages
            "artifacts": {}
        }

    def _generic_execute(self, step: dict) -> dict:
        return {
            "status": "success",
            "output": f"Executed: {step['description']}",
            "artifacts": {}
        }


# ═══════════════════════════════════════════════════════════
# CRITIC AGENT
# ═══════════════════════════════════════════════════════════

class CriticAgent(BaseAgent):
    """
```

```
    Reviews and validates work from other agents.
    Provides verdicts: PASS, RETRY, or REPLAN.
    """

    def __init__(self, bus: MessageBus):
        super().__init__(Role.CRITIC, bus)
        self.review_count = 0

    def review(self, step: dict, result: dict,
               all_context: dict) -> dict:
        """Review a step's result."""
        self.review_count += 1
        self.log(f"Reviewing step {step['id']}: {step['description']}")

        issues = []
        suggestions = []

        # Check for errors
        if result.get("status") != "success":
            issues.append(f"Step failed with status: {result.get('status')}")
            return self._verdict("RETRY", issues, suggestions)

        output = result.get("output", "")

        # Check for empty output
        if not output or len(output.strip()) < 10:
            issues.append("Output is empty or too brief")
            return self._verdict("RETRY", issues, suggestions)

        # Action-specific checks
        action = step.get("action", "")

        if action == "code":
            issues, suggestions = self._review_code(output, result)
        elif action == "test":
            issues, suggestions = self._review_tests(output, result)
        elif action == "analyze":
            issues, suggestions = self._review_analysis(output)
        elif action == "summarize":
            issues, suggestions = self._review_summary(output, all_context)

        # Determine verdict
        critical_issues = [i for i in issues if "CRITICAL" in i.upper()]

        if critical_issues:
            verdict = "REPLAN" if self.review_count > 2 else "RETRY"
        elif issues:
            verdict = "RETRY" if self.review_count <= 2 else "PASS"
        else:
            verdict = "PASS"

        return self._verdict(verdict, issues, suggestions)

    def _review_code(self, output: str, result: dict) -> tuple:
        """Review generated code."""
        issues = []
```

```
        suggestions = []
        code = result.get("artifacts", {}).get("code", output)

        # Check for common issues
        if "eval(" in code:
            issues.append("CRITICAL: Use of eval() — security risk")

        if "password" in code.lower() and "=" in code:
            if "os.environ" not in code and "getenv" not in code:
                issues.append("HIGH: Possible hardcoded credentials")

        if "except:" in code:
            issues.append("MEDIUM: Bare except clause — catch specific exceptions")
            suggestions.append("Use specific exception types")

        if "import" not in code:
            issues.append("MEDIUM: No imports found — code may be incomplete")

        if "def " not in code and "class " not in code:
            issues.append("HIGH: No functions or classes — code structure needed")

        if not issues:
            suggestions.append("Code looks clean. Consider adding docstrings.")

        return issues, suggestions

    def _review_tests(self, output: str, result: dict) -> tuple:
        """Review test code."""
        issues = []
        suggestions = []

        test_count = result.get("artifacts", {}).get("test_count", 0)

        if test_count < 3:
            issues.append("MEDIUM: Too few tests — aim for at least 5")

        if "assert" not in output:
            issues.append("HIGH: No assertions found in tests")

        if "edge" not in output.lower() and "error" not in output.lower():
            suggestions.append("Consider adding edge case and error tests")

        return issues, suggestions

    def _review_analysis(self, output: str) -> tuple:
        """Review an analysis output."""
        issues = []
        suggestions = []

        if len(output) < 100:
            issues.append("MEDIUM: Analysis is too brief")

        key_sections = ["scope", "depend", "consider"]
        missing = [s for s in key_sections if s not in output.lower()]
        if missing:
            suggestions.append(f"Consider adding sections about: {', '.join(missing)
```

```
        return issues, suggestions

    def _review_summary(self, output: str, context: dict) -> tuple:
        """Review a PR summary."""
        issues = []
        suggestions = []

        required_sections = ["changes", "testing"]
        for section in required_sections:
            if section not in output.lower():
                issues.append(f"MEDIUM: Missing '{section}' section in summary")

        return issues, suggestions

    def _verdict(self, verdict: str, issues: list,
                 suggestions: list) -> dict:
        """Create a verdict response."""
        emoji = {"PASS": "✅", "RETRY": "🔄", "REPLAN": "❌"}
        self.log(f"Verdict: {emoji.get(verdict, '?')} {verdict}")

        if issues:
            self.log(f"Issues: {issues}")
        if suggestions:
            self.log(f"Suggestions: {suggestions}")

        self.send(
            Role.ORCHESTRATOR,
            f"Verdict: {verdict}",
            "verdict",
            {"verdict": verdict, "issues": issues, "suggestions": suggestions}
        )

        return {
            "verdict": verdict,
            "issues": issues,
            "suggestions": suggestions,
            "confidence": 0.85 if not issues else 0.6
        }


# ═══════════════════════════════════════════════════════════════
# ORCHESTRATOR
# ═══════════════════════════════════════════════════════════════

class Orchestrator:
    """
    Coordinates all agents in the multi-agent system.

    Flow:
    ┌─────────────────────────────────────────────────────┐
    │                  ORCHESTRATOR                       │
    │                                                     │
    │  1. Receive task                                    │
    │  2. Ask Planner to create plan                      │
    │  3. For each step:                                  │
```

```
        │     a. Ask Executor to execute                          │
        │     b. Ask Critic to review                             │
        │     c. Handle verdict (PASS/RETRY/REPLAN)               │
        │  4. Compile final result                                │
        │                                                         │
        └─────────────────────────────────────────────────────────┘
        """

        def __init__(self):
            self.bus = MessageBus()
            self.planner = PlannerAgent(self.bus)
            self.executor = ExecutorAgent(self.bus)
            self.critic = CriticAgent(self.bus)
            self.guardrails = MultiAgentGuardrails()

        def solve(self, task: str, verbose: bool = True) -> dict:
            """
            Solve a task using the multi-agent system.
            """
            if verbose:
                print(f"\n{'='*60}")
                print(f"  MULTI-AGENT SYSTEM")
                print(f"  Task: {task[:80]}")
                print(f"{'='*60}")

            # Phase 1: Planning
            if verbose:
                print(f"\n{'─'*40}")
                print(f"  PHASE 1: PLANNING")
                print(f"{'─'*40}")

            plan = self.planner.create_plan(task)

            # Phase 2: Execution with Critic validation
            if verbose:
                print(f"\n{'─'*40}")
                print(f"  PHASE 2: EXECUTION")
                print(f"{'─'*40}")

            completed_steps = []
            all_results = {}

            for step in plan["steps"]:
                # Check guardrails
                ok, msg = self.guardrails.check_turn(step.get("agent", "executor"))
                if not ok:
                    if verbose:
                        print(f"\n  ⛔ Guardrail: {msg}")
                    break

                if verbose:
                    print(f"\n  {'─'*30}")
                    print(f"  Step {step['id']}: {step['description']}")
                    print(f"  {'─'*30}")

                # Execute
```

```
            result = self.executor.execute_step(step)
            all_results[step["id"]] = result

            # Critic review (if step is assigned to critic, skip self-review)
            if step.get("agent") == "critic":
                # Critic is both executor and reviewer for review steps
                verdict_result = self.critic.review(
                    step, result, self.executor.context
                )
            else:
                verdict_result = self.critic.review(
                    step, result, self.executor.context
                )

            verdict = verdict_result["verdict"]

            if verdict == "PASS":
                step["status"] = "done"
                completed_steps.append(step["id"])

            elif verdict == "RETRY":
                ok, msg = self.guardrails.check_retry(str(step["id"]))
                if ok:
                    if verbose:
                        print(f"  🔄 Retrying step {step['id']}...")
                    # Re-execute
                    result = self.executor.execute_step(step)
                    all_results[step["id"]] = result
                    step["status"] = "done"
                    completed_steps.append(step["id"])
                else:
                    if verbose:
                        print(f"  ⛔ {msg}")
                    step["status"] = "failed"

            elif verdict == "REPLAN":
                ok, msg = self.guardrails.check_replan()
                if ok:
                    if verbose:
                        print(f"\n  📋 REPLANNING...")
                    feedback = "; ".join(verdict_result.get("issues", []))
                    plan = self.planner.revise_plan(
                        plan, feedback, completed_steps
                    )
                    # Continue with revised plan
                    break
                else:
                    if verbose:
                        print(f"  ⛔ {msg}")

        # Phase 3: Compile results
        if verbose:
            print(f"\n{'─'*40}")
            print(f"  PHASE 3: RESULTS")
            print(f"{'─'*40}")
```

```
        final_result = self._compile_results(task, plan, all_results)

        if verbose:
            print(f"\n{final_result['summary']}")

            print(f"\n{'—'*40}")
            print(f"  STATISTICS")
            print(f"{'—'*40}")
            print(f"  Steps completed: {len(completed_steps)}/{len(plan['steps'])}")
            print(f"  Agent turns: {json.dumps(dict(self.guardrails.agent_turn_count
            print(f"  Retries: {sum(self.guardrails.retry_counts.values())}")
            print(f"  Replans: {self.guardrails.replan_count}")
            print(f"  Messages exchanged: {len(self.bus.messages)}")

        return final_result

    def _compile_results(self, task: str, plan: dict,
                         results: dict) -> dict:
        """Compile all agent outputs into final result."""
        artifacts = {}
        outputs = []

        for step_id, result in results.items():
            if isinstance(result, dict):
                if "output" in result:
                    outputs.append(result["output"])
                if "artifacts" in result:
                    artifacts.update(result["artifacts"])

        summary = f"\n{'='*60}\n"
        summary += f"📋 Task: {task}\n"
        summary += f"{'='*60}\n"

        for i, output in enumerate(outputs, 1):
            if len(output) > 300:
                output = output[:300] + "...\n(truncated)"
            summary += f"\n📌 Output {i}:\n{output}\n"

        return {
            "task": task,
            "plan": plan,
            "results": results,
            "artifacts": artifacts,
            "summary": summary,
            "message_log": self.bus.get_conversation_log()
        }


# ═══════════════════════════════════════════════════════════════
# DEMO
# ═══════════════════════════════════════════════════════════════

def demo():
    """Run multi-agent demo."""

    # Demo 1: Engineering task (Jira-like ticket)
```

```
    print("\n" + "="*60)
    print("  DEMO 1: Engineering Task")
    print("="*60)

    orchestrator = Orchestrator()
    orchestrator.solve(
        "Implement a REST API for task management with CRUD operations. "
        "The API should support creating, reading, updating, and deleting tasks. "
        "Each task has a title, description, and status (todo/in_progress/done). "
        "Include input validation and proper error handling."
    )

    # Demo 2: Analysis task
    print("\n\n" + "="*60)
    print("  DEMO 2: Analysis Task")
    print("="*60)

    orchestrator2 = Orchestrator()
    orchestrator2.solve(
        "Analyze the best approach for implementing authentication "
        "in our new microservices architecture."
    )


if __name__ == "__main__":
    demo()
```

Exercise: Extend the System

```
1. ADD A RESEARCHER AGENT
   Create a ResearcherAgent that uses the RAG pipeline from Day 5
   to gather information. The Planner should be able to assign
   research steps to this agent.

2. ADD PARALLEL EXECUTION
   When the plan has independent steps (no dependencies),
   execute them in parallel using threading.

3. ADD AGENT MEMORY
   Each agent should remember outcomes of previous tasks.
   When starting a new task, check if similar tasks were
   solved before and use those learnings.

4. ADD HUMAN-IN-THE-LOOP
   At critical steps (e.g., before deploying, before major
   code changes), pause and ask for human approval before
   continuing.

5. CONNECT TO REAL LLM
```

```
    Replace the simulated responses with actual LLM calls.
    Use GPT-4 for Planner and Critic, GPT-3.5 for Executor.
```

✓ Day 6 Deliverable Checklist

- [ ] Built a 3-agent system (Planner + Executor + Critic)
- [ ] Agents communicate via a message bus
- [ ] Orchestrator coordinates agent turns
- [ ] System handles PASS / RETRY / REPLAN verdicts
- [ ] Guardrails prevent loops and runaway execution
- [ ] Tested with at least 2 different task types
- [ ] Can explain: What are the main multi-agent orchestration patterns?
- [ ] Can explain: What agent roles exist and when to use each?
- [ ] Can explain: How to prevent loops and deadlocks?
- [ ] Can explain: What are the trade-offs of multi-agent vs single-agent?

Next → Day 7: Final Project — Build Your Own Copilot